Portfolio | Morgan Vanvelthem

Table des matières

1. Introduction au projet
2. Gestion de Projet
3. Développement Web Full Stack
4. Base de Données
5. Développement Embarqué
6. Bilan et Perspectives

Introduction au projet

Contexte

Le projet **Smart Campus** est une initiative visant la sobriété énergétique au sein de l'IUT. L'objectif est de développer une application complète permettant de déployer et monitorer des **Systèmes d'Acquisition (SA)**. Ces boîtiers remontent des données (température, CO2) pour permettre à un Chargé de Mission de prendre des décisions éclairées et d'informer les usagers.

Organisation temporelle

Nous sommes une équipe de **5 étudiants**. Le projet s'est déroulé sur un semestre, rythmé par des itérations agiles aux intensités variables:

- **Sprint 0** (Semaines 40-42) : 4.5h TP + 6h autonomie — *Cadrage et lancement*.
- **Sprint 1** (Semaines 45-47) : 1.5 jour par semaine — *Premières fonctionnalités*.
- **Sprint 2** (Semaines 48-50) : 1.5 jour par semaine — *Montée en charge*.
- **Sprint 3** (Semaines 51-03) : 3 semaines à temps plein — *Finalisation et livraison*.

Mes rôles

La méthodologie imposait une rotation des rôles. Mon implication a évolué ainsi :

- **Sprint 0, 1 et 2** : Développeur Web.
- **Sprint 3** : **Product Owner (PO)**. C'est sur cette période intensive de 3 semaines que j'ai pris le leadership de l'organisation.

Gestion de Projet

Vue d'ensemble

En tant que Product Owner sur la phase finale (Sprint 3), j'ai dû piloter l'atterrissage du projet. Mon objectif n'était pas seulement de coder, mais de garantir que ce qui était produit apportait de la valeur au client tout en maintenant une organisation fluide pour l'équipe.

---

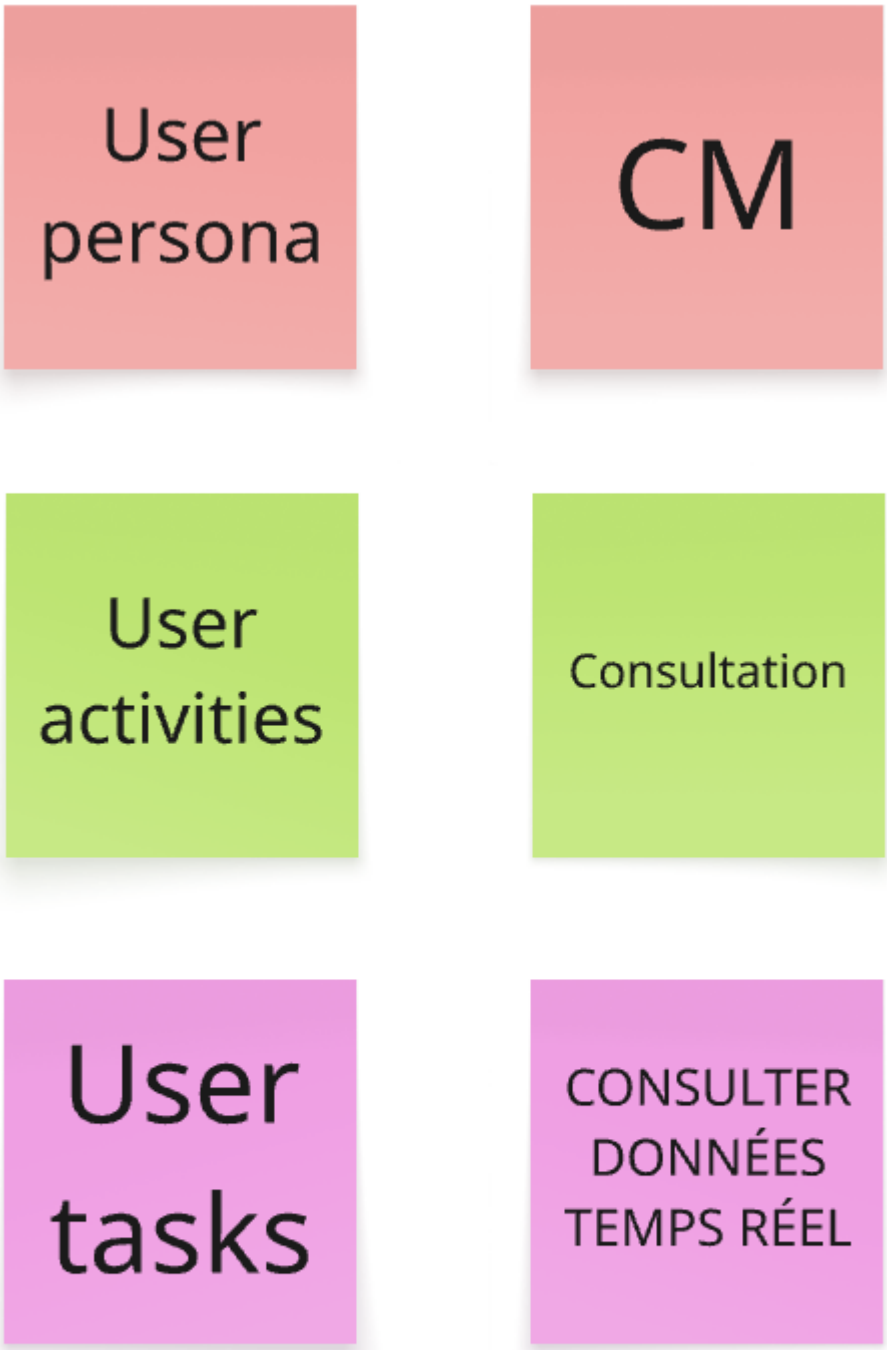
1. Formalisation des besoins (Négociation Client)

Story Mapping

Avant de coder, nous avons formalisé la vision produit via un **Story Mapping**. Cet outil permet de visualiser l'ensemble des fonctionnalités du produit, organisées par parcours utilisateur et par priorité.

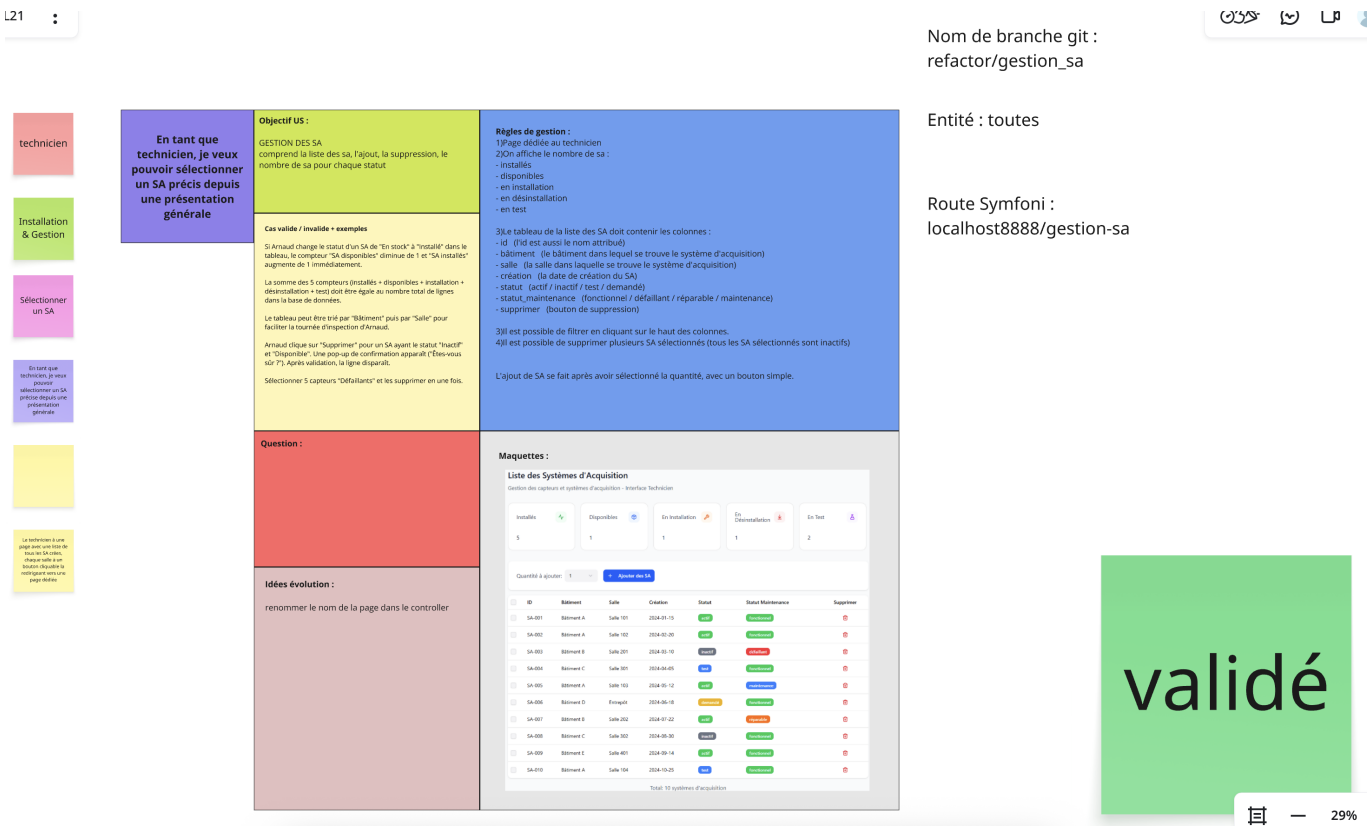
Utilité :

- Avoir une **vision globale** du produit et de ses fonctionnalités
- Identifier le **MVP (Minimum Viable Product)** : les fonctionnalités essentielles pour la première livraison
- Aligner l'équipe et le client sur les priorités



{width="122" height="428"}

En complément de la story mapping on avait ensuite l'exemple mapping (ci-dessous), qui permet d'énumérer les règles de gestions, l'objectif de l'use story et les cas valide / invalide en découlant.



{width="798" height="477"}

Definition of Ready (DOR) et Definition of Done (DOD)

Pour garantir la qualité et éviter les malentendus, nous avons défini des critères clairs et explicites :

Definition of Ready (DOR) - Critères pour qu'un ticket soit prêt à être développé :

```
- [ ] La valeur utilisateur est claire (En tant que... Je veux...)
- [ ] Les critères d'acceptation et tests sont définis
- [ ] Base de données fonctionnelle
- [ ] Les maquettes sont validées par le PO (Exemple / story mapping)
- [ ] Pas de dépendance bloquante connue (manque d'une API ou autre)
```

Definition of Done (DOD) - Critères pour qu'un ticket soit considéré comme terminé :

```
- [ ] Les tests ne remontent aucune erreur
- [ ] Tous les critères d'acceptation de la story sont validés
- [ ] Les cas d'erreurs et messages utilisateurs sont gérés
- [ ] Les tests unitaires sont écrits et passent (Green)
- [ ] Code review validée par le PO
- [ ] Mergé et déployé sur l'environnement de test
```

Ces critères étaient intégrés directement dans les tickets GitLab du kanban sous forme de checklists à cocher.

smart-campus#16

Modifier une Salle

Edit

Open Issue created 2 months ago by **Eduard Golub**

https://miro.com/app/board/uXjVJB0XVDE=/?moveToWidget=3458764647069420311&cot=14

DOR :

- [x] La valeur utilisateur est claire (En tant que... Je veux...).
- [x] Les critères d'acceptation et tests sont définis.
- [x] Base de donnée fonctionnelle
- [x] Les maquettes sont validés par le PO. (Exemple / story mapping)
- [x] Pas de dépendance bloquante connue (manque d'une API ou autre).

DOD :

- [x] Les tests ne remontent aucune erreur.
- [x] Tous les critères d'acceptation de la story sont validés. (règles)
- [x] Les cas d'erreurs et messages utilisateurs sont gérés.
- [x] Les tests unitaires sont écrits et passent (Green).
- [x] Code review validée par le PO.
- [x] Mergé et déployé sur l'environnement de test.

✓ 11 of 11 checklist items completed · Edited 14 hours ago by Simon Plault

0 0 Add design Create merge request

Assignee Edit

Noe Leteurtre

Labels Edit

Priorité : moyenne × Terminer × Type : fonctionnalité ×

{width="750" height="790"}

Le problème de la gestion des SA

Lors des sprints précédents, nous avions imaginé une gestion complexe des capteurs, distinguant leur "position physique" de leur "état technique". Lors d'une revue avec le client (Chargé de mission), j'ai senti que cette complexité créait de la confusion : le client peinait à comprendre comment déclarer un capteur "HS".

Mon action : Plutôt que d'attendre ou de coder aveuglément cette complexité, j'ai profité de l'échange oral pour proposer immédiatement une alternative : un cycle de vie unifié (ex: `En stock – Fonctionnel`, `Installé – HS`). J'ai expliqué ce schéma simplifié en direct.

Résultat : Le client a validé cette approche instantanément car elle correspondait à son usage réel.

Analyse réflexive : Cette intervention a eu un double bénéfice. Pour le **client**, l'application est devenue plus intuitive. Pour l'**équipe**, cela a évité des heures de développement inutile et des conflits de logique dans le code. J'ai appris qu'un bon PO doit savoir challenger le besoin pour le simplifier, et non juste l'accepter passivement.

---

2. Pilotage et Suivi (Méthode Kanban)

Adaptation de l'Issue Board

Nous utilisons GitLab pour suivre nos tâches. Cependant, le tableau par défaut ne suffisait pas à gérer le flux intense du Sprint 3.

Mon action : J'ai restructuré le tableau pour une meilleure lisibilité :

- Création de **Labels hiérarchisés** (Priorité vs Type) pour que l'équipe sache quoi traiter en premier.
- Utilisation des colonnes pour visualiser le flux de validation.
- Intégration des **DOD (Definition of Done)** et **DOR (Definition of Ready)** directement dans les tickets sous forme de checklists à cocher.

Discipline de travail et Vision Globale : Au-delà des outils, j'ai adopté une rigueur personnelle que j'ai tenté d'insuffler à l'équipe :

- **Gestion du flux (WIP) :** Je m'interdis d'ouvrir un nouveau ticket tant que celui sur lequel je travaille n'est pas totalement terminé (DOD validée). Cela évite la dispersion.
- **Veille active :** Je ne me contente pas des réunions pour m'informer. Je consulte régulièrement le tableau pour voir en temps réel sur quoi mes camarades avancent. Cela me permet d'avoir une vision globale constante de la santé du sprint et d'intervenir spontanément si je vois une tâche stagner, sans attendre le point de synchronisation suivant.

Contrôle Qualité : En tant que PO, je me suis imposé une règle : aucun ticket ne passe dans la colonne "Terminé" sans que je n'aie vérifié le code ou la fonctionnalité sur le poste du collègue.

Recul critique (Axe d'amélioration) : C'est ici que j'identifie ma plus grande marge de progression. Je faisais ces vérifications manuellement, en allant voir mes collègues.

- *Le constat :* C'était efficace mais chronophage et non traçable, il m'est arrivé de me perdre et de passer certains tickets dans terminer par erreur.
- *L'apprentissage :* J'ai réalisé *a posteriori* que nous aurions dû utiliser les **Merge Requests (MR)** de GitLab. Cela aurait permis de formaliser ma validation de code, de laisser des commentaires écrits et d'automatiser le processus. C'est une pratique que je mettrai impérativement en place au prochain semestre pour professionnaliser le "Code Review".

Axes d'amélioration : KPIs et Indicateurs

Burndown Chart et Vélocité : Nous n'avons pas utilisé de Burndown Chart ni calculé la vélocité de l'équipe. Les estimations étaient faites **oralement** avant chaque attribution de tâche, sans formalisation en story points.

Ce que cela aurait apporté :

- Une **visualisation objective** de l'avancement du sprint
- Une **capacité de prévision** pour les sprints futurs
- Une **détection précoce** des retards

Plan d'action pour l'avenir : Pour le prochain projet, nous verrons avec l'équipe comment une mise en place des éléments ci dessous aiderais à notre organisation :

- Des estimations en **story points** lors du Sprint Planning
- Un **Burndown Chart** mis à jour quotidiennement
- Un calcul de la **vélocité** pour affiner les prévisions

---

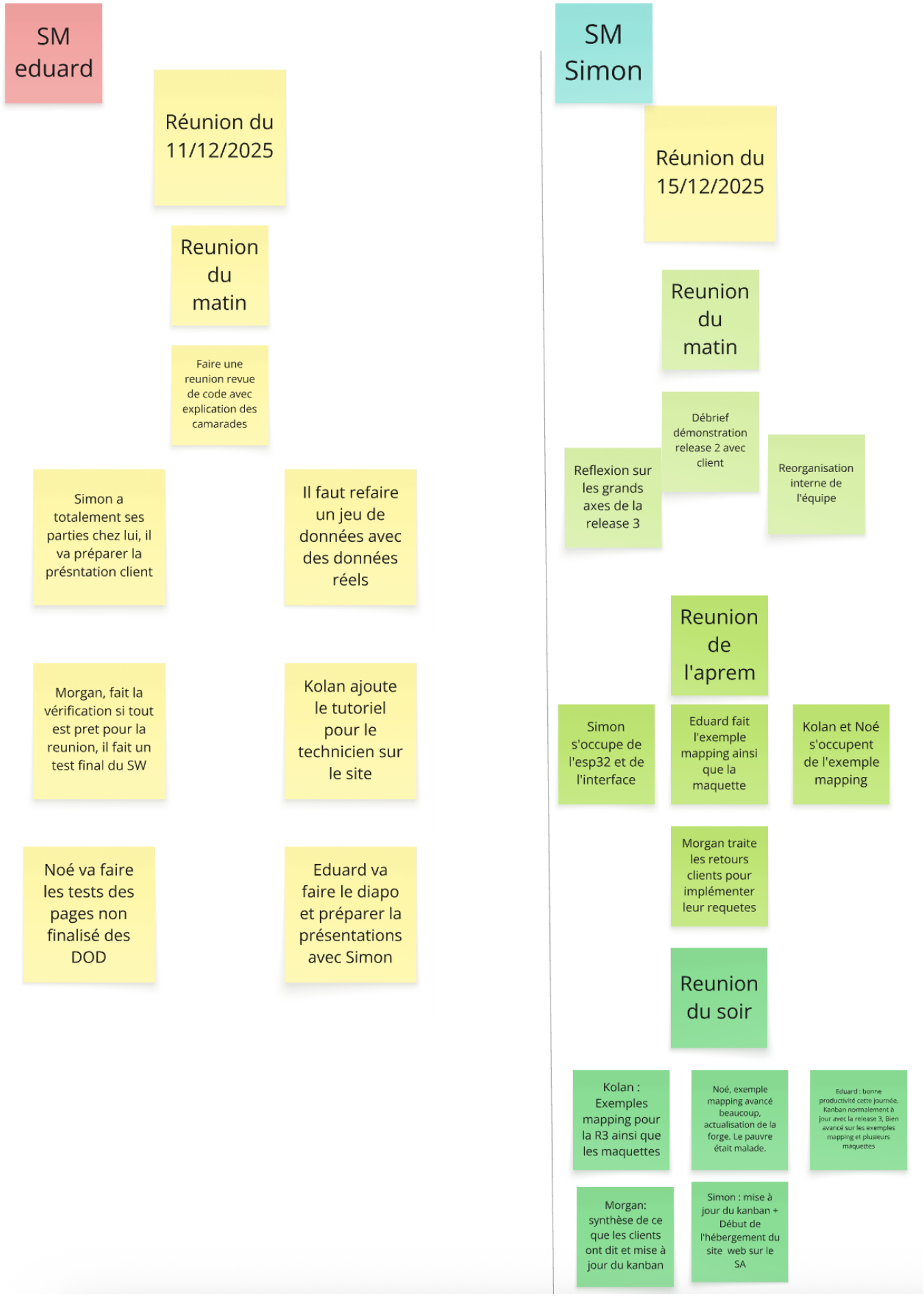
3. Cérémonies Agile et Communication

Daily Meetings (Stand-up)

La fréquence des Daily Meetings a évolué avec l'intensité du projet :

Sprint 3 (temps plein) : Pour maintenir une synchronisation optimale pendant la phase intensive, nous avons mis en place **3 points de synchronisation par jour** :

- **Matin (arrivée)** : Ce que je vais faire aujourd'hui, blocages identifiés
- **14h** : Point d'avancement mi-journée, ajustements si nécessaire
- **Soir (avant départ)** : Ce qui a été fait, ce qui reste, préparation du lendemain



{width="431" height="600"}

Cette cadence soutenue a permis une réactivité maximale et une détection rapide des problèmes.

Sprint Reviews (Démonstrations Client)

Les revues de sprint avec le client suivaient un format structuré :

Déroulement :

1. Présentation des fonctionnalités développées (démo live)
2. Feedback du client sur chaque fonctionnalité
3. **Prise de notes systématique** : demandes, recommandations, améliorations

Bonne pratique adoptée : Nous gardions **traces écrites** de tous les échanges avec le client. Cela permettait de :

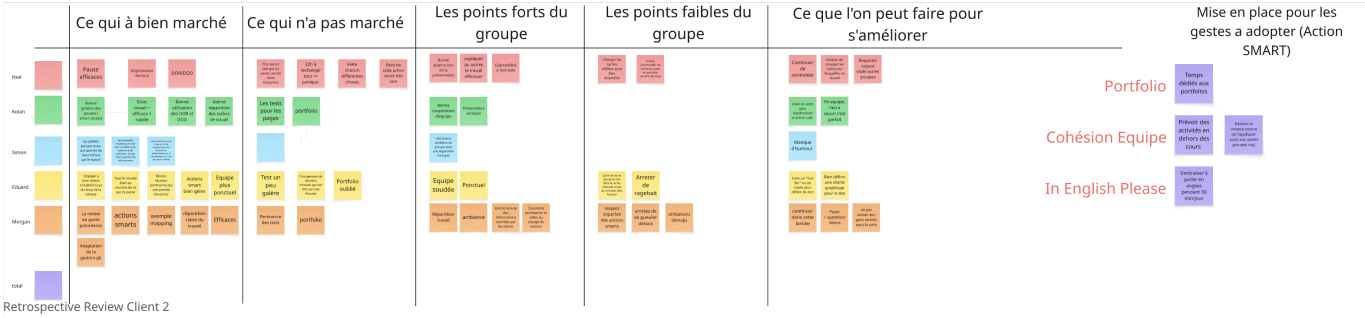
- Ne rien oublier des retours
- Justifier nos choix de priorisation
- Alimenter le backlog avec les nouvelles demandes

Rétrospectives

Les rétrospectives ont été **améliorées au fur et à mesure des sprints**. Nous avons appris à les rendre plus efficaces et à en tirer des actions concrètes.

Évolution de notre pratique :

- Sprint 1 : Rétrospectives basiques, peu structurées
- Sprint 2 : Introduction d'un format (Ce qui a bien marché / À améliorer / Actions)
- Sprint 3 : Rétrospectives productives avec plans d'action suivis



{width="900" height="426"}

---

4. Dynamique d'équipe et Compétences Interpersonnelles

Rôles assumés

Les responsabilités étaient clairement définies et **assumées pleinement** :

Rôle	Responsabilité	Mon implication
Product Owner	Vision produit, priorisation, validation	Sprint 3
Développeurs	Implémentation des fonctionnalités	Sprints 0-2
Équipe	Auto-organisation, entraide	Tous les sprints

Auto-organisation et Autonomie

L'équipe fonctionnait de manière **autonome** :

- Résolution des problèmes techniques sans attendre l'enseignant
- Répartition des tâches décidée collectivement
- Entraide spontanée entre les membres

Climat de travail et Cohésion

Un cadre de travail **agréable et propice à la productivité** a été maintenu tout au long du projet :

Équilibre travail / détente :

- Pauses régulières pour souffler
- Moments de convivialité (jeux de cartes) permettant de discuter de manière informelle du projet
- Ces moments ont renforcé la **cohésion d'équipe** et facilité la communication

Entraide spontanée :

- Quand un membre bloquait, les autres venaient naturellement aider
- Pas de silos : les développeurs Front et Back communiquaient régulièrement

Organisation de l'équipe

Le Sprint 3 étant à temps plein, le risque de désorganisation était élevé.

Mon action : J'ai animé les rituels agiles (Daily Meetings) en m'assurant que la parole circulait. J'ai veillé à ce que les développeurs "Back" et "Front" se parlent avant de commencer une fonctionnalité pour s'accorder sur les données.

Exemple concret : Suite à la simplification du modèle SA (mentionnée plus haut), j'ai dû réexpliquer la nouvelle logique à l'équipe. J'ai utilisé le diagramme d'états comme support visuel pour m'assurer que tout le monde (Dev Web et Dev BD) partait sur la même base, réduisant ainsi les quiproquos techniques.

Developpement Web Full Stack

Vue d'ensemble

Sur ce projet, j'ai été responsable du développement du module "Gestion de Parc" destiné aux techniciens. J'ai travaillé sur la stack technique imposée : **Symfony 6** (Framework PHP), **Twig** (Moteur de template) et

MariaDB (Base de données).

Mon défi principal n'était pas seulement de faire fonctionner l'application, mais de garantir une architecture maintenable et robuste.

---

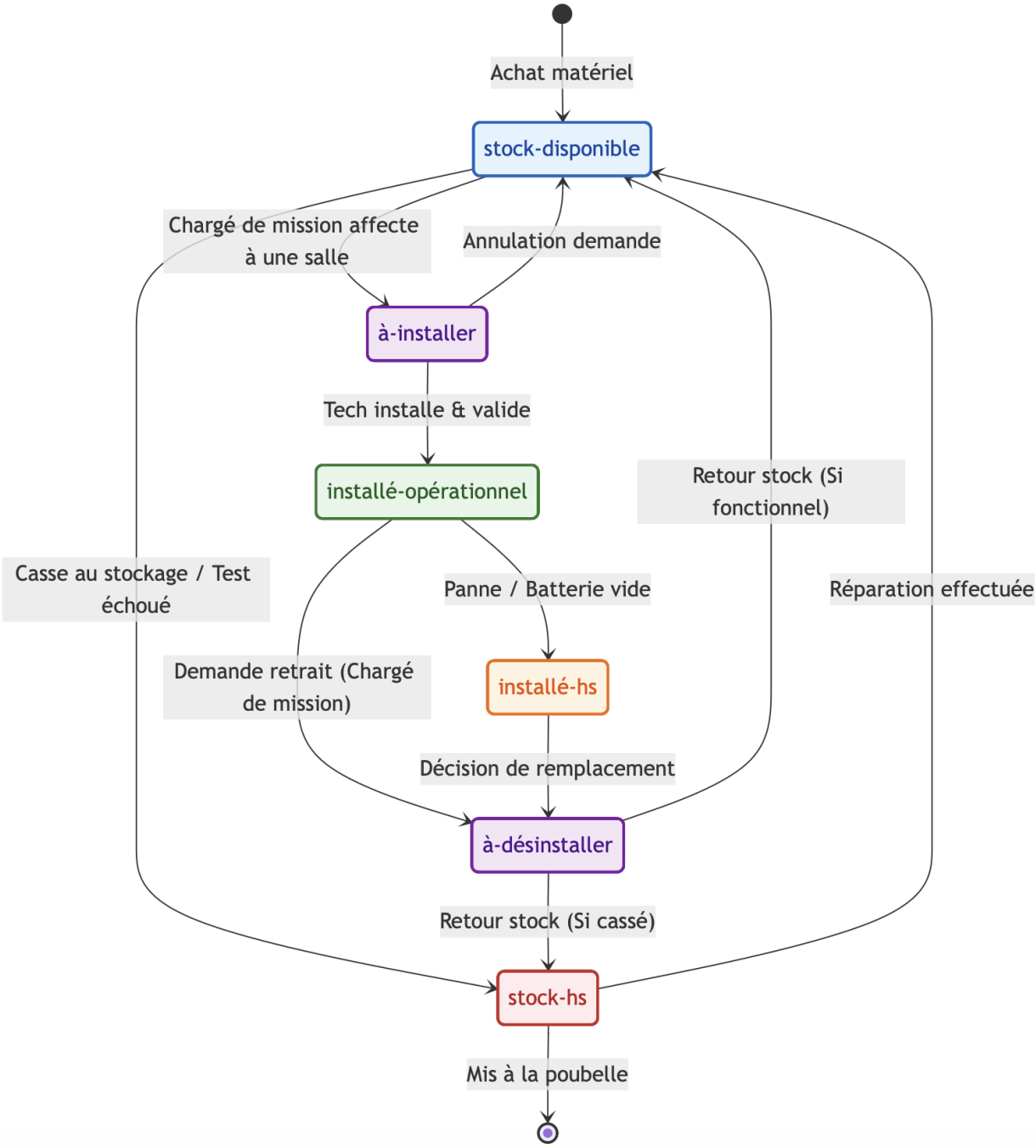
1. Conception : De la complexité à l'efficacité

Le défi de la modélisation

Initialement, nous avions conçu une base de données très normalisée distinguant la "Position" d'un boîtier (Stock, Salle) de son "État Technique" (Neuf, HS, En panne).

Problème rencontré : Lors du développement, je me suis rendu compte que cette séparation multipliait les conditions dans le code (`if position == stock AND etat == hs`). Cela rendait la maintenance complexe et augmentait le risque d'incohérences (ex: un boîtier "En salle" mais "En stock" par erreur).

Ma solution (Refactoring) : J'ai proposé et implémenté une **Machine à États Finis** unifiée. J'ai fusionné les concepts en une seule propriété `status` (ex: `stock_operationnel`, `installe_hs`).



{width="595" height="600"}

Analyse réflexive : Ce choix d'architecture a réduit la complexité algorithmique. J'ai appris qu'une bonne base de données n'est pas celle qui est la plus "théoriquement pure", mais celle qui correspond le mieux à la logique métier réelle de l'application.

---

2. Routes et Navigation

Organisation des routes

Les routes de l'application sont définies via les **attributs PHP 8** directement dans les contrôleurs. Pour les pages nécessitant d'afficher des détails d'une entité spécifique (comme `info-sa` ou `info-salle`), j'ai utilisé des **routes paramétrées** avec des identifiants.

Exemple : Route paramétrée pour les détails d'un SA

```
#[Route('/info-sa/{id}', name: 'app_info_sa', methods: ['GET', 'POST'])]
public function index(
    int $id,
    SystemeAcquisitionRepository $systemeAcquisitionRepository,
    Request $request,
    EntityManagerInterface $em,
    FormFactoryInterface $formFactory
): Response
{
    $sa = $systemeAcquisitionRepository->find($id);

    if (!$sa) {
        throw $this->createNotFoundException('Le système d\'acquisition #'
. $id . ' n\'existe pas !');
    }

    // ... traitement du formulaire de renommage
}
```

Points clés :

- L'identifiant `{id}` est automatiquement injecté comme paramètre de la méthode
- La gestion d'erreur 404 est assurée si l'entité n'existe pas
- Les méthodes HTTP autorisées sont explicitement définies (`GET`, `POST`)

---

3. Architecture & Qualité Logicielle

Séparation des responsabilités (Le Service)

Au début du projet, mon Contrôleur (`SystemeAcquisitionController.php`) gérait tout : les routes, la création des entités, la validation et l'affichage. Il devenait difficile à lire et à maintenir.

Mon action : Pour respecter le principe de responsabilité unique (Single Responsibility Principle), j'ai extrait la logique métier (comme la création en masse de SA) dans un Service dédié : `SystemeAcquisitionManager`.

Extrait de code (Service) :

```
namespace App\Service\SystemeAcquisition;

/**
 * Service de gestion des Systèmes d'Acquisition
 * Centralise toute la logique métier liée aux SA
 */
class SystemeAcquisitionManager
{
    private EntityManagerInterface $em;

    public function __construct(EntityManagerInterface $em)
```

```
    {
        $this->em = $em;
    }

    /**
     * Crée plusieurs systèmes d'acquisition en stock.
     *
     * @param int $quantite Nombre de SA à créer
     * @return array{success: bool, message: string} Résultat de la
création
     */
    public function createSystemesAcquisition(int $quantite): array
    {
        // Validation
        if ($quantite < 1 || $quantite > 100) {
            return [
                'success' => false,
                'message' => 'La quantité doit être entre 1 et 100.'
            ];
        }

        // Création des SA
        for ($i = 0; $i < $quantite; $i++) {
            $sa = new SystemeAcquisition();
            $sa->setDateCreation(new \DateTime());
            $sa->setEtat('stock-disponible');
            $this->em->persist($sa);
        }

        $this->em->flush();

        $message = $quantite === 1
            ? 'Un nouveau SA a été ajouté au stock.'
            : "$quantite nouveaux SA ont été ajoutés au stock.";

        return [
            'success' => true,
            'message' => $message
        ];
    }

    /**
     * Change l'état d'un système d'acquisition.
     *
     * @param SystemeAcquisition $sa Le SA à modifier
     * @param string $nouvelEtat Le nouvel état
     * @return array{success: bool, message: string} Résultat de la
modification
     */
    public function changeEtat(SystemeAcquisition $sa, string
$nouvelEtat): array
    {
        $allowedEtats = [
            'installé-opérationnel',
```

```
            'installé-hs',
            'stock-disponible',
            'stock-hs',
            'à-installer',
            'à-désinstaller'
        ];

        if (!in_array($nouvelEtat, $allowedEtats)) {
            return [
                'success' => false,
                'message' => 'État invalide. États autorisés : ' .
implode(', ', $allowedEtats)
            ];
        }

        $sa->setEtat($nouvelEtat);
        $this->em->flush();

        return [
            'success' => true,
            'message' => 'État mis à jour avec succès.'
        ];
    }
}
```

Points clés du service :

- **Validation métier centralisée** : les règles (états autorisés, contraintes de suppression) sont au même endroit
- **Retour standardisé** : toutes les méthodes retournent `['success' => bool, 'message' => string]`
- **Délégation au Repository** : les requêtes optimisées sont dans le Repository (voir section Base de Données)

Gain : Le contrôleur est désormais "léger" : il se contente d'appeler `createSystemesAcquisition($qty)` sans se soucier des détails d'implémentation. Ce service est réutilisable (par exemple via une commande console) et testable isolément.

Extrait de code (Contrôleur optimisé) :

Voici comment le contrôleur `GestionSaController` délègue toute la logique au service :

```
final class GestionSaController extends AbstractController
{
    public function __construct(
        private SystemeAcquisitionManager $saManager
    ) {}

    /**
     * Change the state of a specific SA.
     *
```

```
     * @param SystemeAcquisition $sa
     * @param string $etat
     * @return Response
     */
    #[Route('/gestion-sa/change-etat/{id}/{etat}', name:
'app_gestion_sa_change_etat')]
    public function changeEtat(SystemeAcquisition $sa, string $etat):
Response
    {
        $result = $this->saManager->changeEtat($sa, $etat);

        if ($result['success']) {
            $this->addFlash('success', $result['message']);
        } else {
            $this->addFlash('error', $result['message']);
        }

        return $this->redirectToRoute('app_gestion_sa');
    }

    /**
     * Delete a specific SA.
     *
     * @param SystemeAcquisition $sa
     * @return Response
     */
    #[Route('/gestion-sa/delete/{id}', name: 'app_gestion_sa_delete')]
    public function delete(SystemeAcquisition $sa): Response
    {
        $result = $this->saManager->deleteSystemeAcquisition($sa);

        if ($result['success']) {
            $this->addFlash('success', $result['message']);
        } else {
            $this->addFlash('error', $result['message']);
        }

        return $this->redirectToRoute('app_gestion_sa');
    }
}
```

Résultat :

- Le contrôleur ne fait que **coordonner** : recevoir la requête, appeler le service, retourner la réponse
- Toute la **logique métier** (validation des états, règles de suppression, calcul des statistiques) est dans le service
- Le code est **testable** : on peut mocker le service pour tester le contrôleur isolément

Industrialisation et Robustesse (Utilisation des composants Symfony)

Au-delà de mes développements propres, j'ai dû intégrer et utiliser les briques techniques mises en place par l'équipe pour garantir la sécurité et la performance.

Validation des données (Forms) : Pour la génération de stock, j'ai utilisé le système de Formulaires de Symfony plutôt que de traiter les superglobales `$_POST` manuellement.

- *Avantage :* La sécurité (CSRF) et la validation sont gérées centralement.
- *Trace :* J'utilise la classe `SaGenerationType` qui impose, par exemple, une contrainte stricte sur la quantité (entre 1 et 50) avant même que je ne traite la donnée dans mon contrôleur.

```
// Extrait de SaGenerationType utilisé dans mon module




$builder->add('quantite', IntegerType::class, [
    'label' => 'Quantité à générer',
    'constraints' => [
        new Range(['min' => 1, 'max' => 50]),
    ],
]);
```

Fiabilité par les Tests

Pour garantir la robustesse de cette logique métier, j'ai mis en place des tests automatisés avec **PHPUnit**. Cela me permet de vérifier que les règles de gestion sont toujours respectées, même après une modification du code.

Tests implémentés pour `GestionSaController` :

```
✔ Page gestion sa est accessible
✔ Page affiche la liste des sa
✔ Ajout sa avec succes
✔ Ajout un seul sa
✔ Ajout sa echec quantite invalide min
✔ Ajout sa echec quantite invalide max
✔ Changement etat sa
✔ Changement vers etats autorises
✔ Changement etat non autorise
✔ Get ne cree pas de sa
✔ Affichage differents etats
✔ Vue memorisee apres ajout
✔ Date creation correctement definie
```

Couverture des tests (PHPUnit Coverage) :

Métrique	Couverture	Détail
Lignes	77.65%	1887 / 2430

Métrique	Couverture	Détail
Fonctions	75.35%	269 / 357
Classes	46.34%	38 / 82

Analyse :

- La couverture globale de **77.65%** est proche de l'objectif industriel de 80%
- Les tests couvrent les **cas nominaux** (ajout réussi, changement d'état) et les **cas limites** (quantité min/max invalide, état non autorisé)
- La couverture des classes (46%) est plus faible car certaines classes utilitaires ou entités simples ne nécessitent pas de tests dédiés

Axe d'amélioration : Pour atteindre les 80%, il faudrait ajouter des tests sur les autres contrôleurs et améliorer la couverture des services secondaires.

---

4. Interface et Ergonomie

Restitution dynamique des données

Pour le technicien, l'interface doit être un outil de travail rapide. Il doit voir l'état du parc en un coup d'œil.

Mon action : J'ai utilisé la puissance de **Twig** pour créer une interface conditionnelle. Plutôt que de simples textes, j'ai mis en place un code couleur visuel (Badges) et un menu d'actions contextuel qui change selon l'état du boîtier.

Badges d'état conditionnels (extrait du template `gestion_sa.html.twig`) :

```
{% set etat = sa.getEtat() %}
{% if etat == 'installé-opérationnel' %}
    <span class="status-badge operationnel">
        <svg>...</svg> opérationnel
    </span>
{% elseif etat == 'installé-hs' or etat == 'stock-hs' %}
    <span class="status-badge hs">
        <svg>...</svg> HS
    </span>
{% elseif etat == 'stock-disponible' %}
    <span class="status-badge disponible">
        <svg>...</svg> disponible
    </span>
{% elseif etat == 'à-installer' %}
    <span class="status-badge a-installer">
        <svg>...</svg> à installer
    </span>
{% endif %}
```

Chaque état a sa couleur CSS associée (vert pour opérationnel, rouge pour HS, orange pour interventions).

Menu d'actions contextuel :

```
{# Actions différentes selon l'état du SA #}
{% if sa.getEtat() == 'à-installer' or sa.getEtat() == 'à-désinstaller' %}
    <div class="action-menu-item" onclick="viewDemande({{ sa.id }})">Voir
demande</div>
{% endif %}

{% if sa.getEtat() == 'installé-hs' %}
    <div class="action-menu-item" onclick="changeEtat({{ sa.id }},
'installé-opérationnel')">Marquer opérationnel</div>
{% elseif sa.getEtat() == 'stock-disponible' %}
    <div class="action-menu-item" onclick="changeEtat({{ sa.id }}, 'stock-
hs')">Marquer HS</div>
{% endif %}

{# Suppression uniquement possible pour les SA en stock #}
{% if sa.getEtat() == 'stock-disponible' or sa.getEtat() == 'stock-hs' %}
    <div class="action-menu-item delete" onclick="deleteSa({{ sa.id
}})">Supprimer</div>
{% endif %}
```

Résultat : Le technicien ne voit que les actions possibles (on ne lui propose pas de "Supprimer" un boîtier qui est "Installé"). Cela réduit la charge mentale de l'utilisateur et prévient les erreurs de manipulation.

Responsive Design (partiel)

L'interface inclut des **media queries** pour s'adapter aux différentes tailles d'écrans :

```
@media (max-width: 1024px) {
    .stats-grid {
        grid-template-columns: 1fr;
    }
    table { font-size: 0.875rem; }
    th, td { padding: 12px; }
}
```

Niveau actuel : Optimisé pour les écrans de bureau et tablettes (> 768px).

Axe d'amélioration : Le responsive **mobile** (< 768px) n'a pas été une priorité car le public cible (techniciens) utilise principalement des postes fixes. Pour une version 2.0, une approche **mobile-first** ou l'utilisation d'un framework CSS (Bootstrap/Tailwind) garantirait une meilleure expérience sur tous les appareils.

---

5. Compréhension des Events & Listeners (Architecture Événementielle)

Contexte du projet

Lors de l'analyse de notre architecture, j'ai exploré le composant **EventDispatcher** de Symfony pour évaluer s'il pouvait améliorer notre code. Après étude, nous n'avons pas identifié de cas d'usage où cette architecture serait réellement nécessaire dans notre projet Smart Campus.

Raisons :

- Les changements d'état des boîtiers se font **manuellement** par les techniciens via l'interface
- Nous n'avons pas de processus automatisés nécessitant des réactions en chaîne (pas d'envoi d'email automatique, pas de notifications temps réel)
- La logique métier actuelle reste simple et ne nécessite pas de découplage supplémentaire

Compréhension du pattern

Néanmoins, j'ai étudié ce pattern pour comprendre son rôle et son utilité dans des contextes plus complexes.

Le principe : L'objectif des Events est de **découpler l'action de ses conséquences**. Au lieu qu'un service gère directement toutes les réactions à une action (envoi d'email, log, notification), on émet un "événement" et des "listeners" y réagissent indépendamment.

Exemple théorique appliqué au Smart Campus :

Si nous devions automatiser l'envoi d'alertes lors d'un changement d'état critique, voici comment j'aurais implémenté ce pattern :

1. **Création d'un événement personnalisé :**

```php
// src/Event/SystemeAcquisitionStatusChangedEvent.php
class SystemeAcquisitionStatusChangedEvent
{
    private SystemeAcquisition $systeme;
    private string $oldStatus;
    private string $newStatus;

    public function __construct($systeme, $oldStatus, $newStatus) {
        $this->systeme = $systeme;
        $this->oldStatus = $oldStatus;
        $this->newStatus = $newStatus;
    }

    public function getSysteme() { return $this->systeme; }
    public function getNewStatus() { return $this->newStatus; }
}
```

2. **Dispatch de l'événement :** Dans le service, après la mise à jour en base, on émettrait l'événement :

```php
// Dans SystemeAcquisitionManager
$oldStatus = $sa->getEtat();
$sa->setEtat($newStatus);
```

```
$this->em->flush();

// Émission de l'événement (sans savoir qui va réagir)
$event = new SystemeAcquisitionStatusChangedEvent($sa, $oldStatus,
$newStatus);
$this->eventDispatcher->dispatch($event);
```

3. **Création d'un Listener :** Un `MaintenanceAlertSubscriber` écouterait cet événement et réagirait si nécessaire :

```
class MaintenanceAlertSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents() {
        return [SystemeAcquisitionStatusChangedEvent::class =>
'onStatusChanged'];
    }

    public function
onStatusChanged(SystemeAcquisitionStatusChangedEvent $event) {
        if ($event->getNewStatus() === 'HS') {
            $this->mailer->send("Alerte : SA HS détecté !");
        }
    }
}
```

Avantage de cette architecture :

- Respect du principe **Open/Closed** : ajouter une nouvelle réaction ne nécessite pas de modifier le service existant
- **Testabilité** : chaque listener peut être testé indépendamment
- **Maintenabilité** : chaque listener a une responsabilité unique et claire

Conclusion

Bien que non implémentés dans ce projet (car non nécessaires), je comprends le rôle des Events et Listeners et saurais les utiliser dans un contexte où des actions automatisées ou des réactions multiples seraient requises.

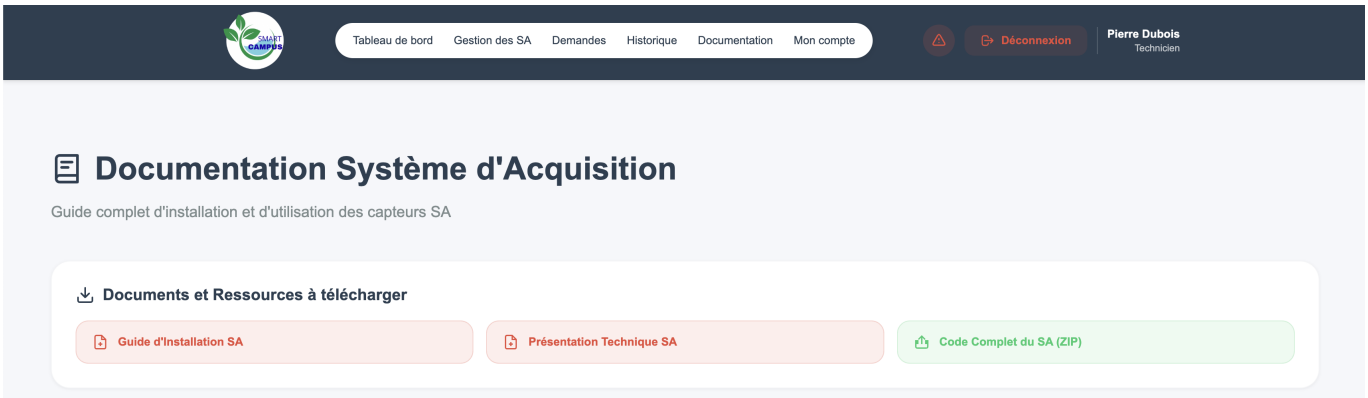
---

6. Documentation

Documentation Utilisateur (Technicien)

Une page de documentation dédiée a été intégrée à l'application pour les techniciens. Elle regroupe :

- **Guide d'installation des SA** : Procédure pas à pas pour installer physiquement un boîtier
- **Présentation technique des SA** : Spécifications matérielles et fonctionnement
- **Code complet des SA** : Référence pour le débogage ou la maintenance



{width="900" height="260"}

Documentation Développeur

La documentation technique est disponible sur le dépôt Git :

- **README principal** : Instructions d'installation et de configuration du projet
- **Documentation des entités** : Description du modèle de données

---

7. Architecture en couches & Repositories

L'architecture du projet respecte une **séparation en couches** claire :

Couche	Responsabilité	Exemple
Controller	Réception des requêtes, coordination	`GestionSaController`
Service	Logique métier	`SystemeAcquisitionManager`
Repository	Accès aux données, requêtes optimisées	`SystemeAcquisitionRepository`
Entity	Modèle de données	`SystemeAcquisition`

Repositories optimisés : Les requêtes complexes (statistiques, agrégations) sont centralisées dans les Repositories avec le **QueryBuilder** de Doctrine.

---

8. Fonctionnalités Avancées : Périmètre de mon travail

Commandes Symfony (Console)

Les commandes Symfony (`php bin/console ...`) permettent d'exécuter des tâches en ligne de commande (imports de données, maintenance, etc.).

Mon implication : Je n'ai pas développé de commandes console personnalisées dans ce projet. Les commandes utilisées sont celles fournies par Symfony et Doctrine (migrations, fixtures).

API Externes

Le projet Smart Campus utilise deux liaisons API :

- **API ↔️ Site Web** : Communication entre l'interface et le backend
- **API ↔️ SA (Boîtiers)** : Transmission des données capteurs

Mon implication : Je n'ai pas travaillé sur ces intégrations API cette partie a été développée par d'autres membres de l'équipe.

Configuration par Variables d'Environnement (.env)

Le projet utilise bien les bonnes pratiques Symfony pour la configuration :

```
# Base de données (Doctrine)
DATABASE_URL="mysql://user:password@db:3306/smart_campus"

# Service de mail (Mailer)
MAILER_DSN=smtp://...@smtp.gmail.com:587

# Credentials API externe
API_CAPTURE_USERNAME=l2eq1
API_CAPTURE_PASSWORD=********
```

Avantages de cette approche :

- Les secrets ne sont pas versionnés (fichiers `.env.local` ignorés par Git)
- Configuration différente par environnement (dev, prod)
- Facilité de déploiement

Mon implication : La mise en place du Mailer et des credentials API a été faite par d'autres membres de l'équipe. J'ai cependant compris et utilisé ce système de configuration dans mes développements.

Cache

Aucune stratégie de cache spécifique n'a été implémentée dans ce projet. C'est un **axe d'amélioration** potentiel pour optimiser les performances en cas de montée en charge (cache des statistiques du dashboard par exemple).

Developpement Embarque

Vue d'ensemble

L'aspect matériel du projet repose sur un microcontrôleur **ESP32** connecté à divers capteurs (SGP30, DHT22) et actionneurs (Écran OLED).

Transparence sur ma contribution : Je tiens à être honnête : ma contribution personnelle sur la partie embarquée représente environ **5% du travail total**. Un membre de l'équipe, expert en ESP32, a réalisé la majeure partie du développement. Cependant, j'ai pu découvrir ce domaine lors des **TP de découverte** où nous avons testé différents composants (capteurs, écran OLED), et j'ai contribué à une fonctionnalité spécifique.

---

1. Ma contribution : Sobriété Numérique (Bouton On/Off Écran)

Le problème énergétique

Laisser l'écran OLED allumé 24h/24 dans une salle vide est un gaspillage d'énergie. J'ai ajouté un **bouton physique** pour permettre l'extinction manuelle de l'écran.

Défi technique (Anti-rebond) : Un bouton mécanique génère des parasites électriques ("rebonds"). J'ai codé un filtre logiciel dans une tâche FreeRTOS dédiée pour stabiliser le signal.

Extrait de code (Gestion Tâche FreeRTOS) :

```
void TaskDisplayData(void *pvParameters) {
    for (;;) {
        // ... (Lecture bouton)
        // Logique Anti-Rebond : On attend 50ms de stabilité
        if ((millis() - lastDebounceTime) > debounceDelay) {
            screenOn = !screenOn;
            if (screenOn) display.displayOn();
            else display.displayOff(); // Coupe l'alimentation de l'écran
        }
        vTaskDelay(pdMS_TO_TICKS(100)); // Gestion multitâche
    }
}
```

Apport : Cette fonctionnalité répond aux enjeux de **sobriété énergétique** du projet Smart Campus.

---

2. Compréhension de l'architecture du SA

Bien que n'ayant pas développé la majorité du code, j'ai acquis une **compréhension globale** de l'architecture grâce aux TP et au travail d'équipe.

Architecture modulaire du projet

Le code est organisé de manière **modulaire** (un fichier par fonctionnalité) :

Fichier	Responsabilité
`main.cpp`	Point d'entrée, initialisation FreeRTOS
`DHT.cpp`	Lecture capteur température/humidité
`co2.cpp`	Lecture capteur CO2 (SGP30)
`ecran.cpp`	Affichage OLED
`Wifi.cpp`	Connexion WiFi
`api.cpp`	Communication avec l'API
`credentials.cpp`	Gestion des identifiants

Fichier	Responsabilité
`globals.h`	Variables globales partagées

Utilisation de Git

Commits clairs avec convention de nommage :

```
[feature] Ajout des infos (co2, humidité...) en direct sur le site
[feat](ecran.cpp): Suppression du débug et nettoyage du code
[fix] Modification de l'architecture des fichiers
[refractor] Reconnexion sauvegarde SSID, user et mdp
```

Branches thématiques : Personnellement, j'ai utilisé des branches pour mes développements. Les collègues travaillant seuls sur le SA n'en avaient pas besoin (pas de conflit possible).

Documentation : Une documentation technique est disponible sur le dépôt Git et intégrée à l'application web pour les techniciens.

---

3. Fonctionnement global du SA (Ce que j'ai compris)

État du produit

Le SA fonctionne : Acquisition des données et transmission à l'API opérationnelles

Limitation connue : La reconnexion automatique en cas de perte WiFi n'a pas pu être finalisée malgré de multiples tentatives (travail réalisé par un collègue, pas par moi).

Gestion des erreurs

La gestion des erreurs (capteur déconnecté, etc.) n'a pas été implémentée. Le temps disponible a été consommé par les tentatives de résolution du problème de reconnexion WiFi.

Axe d'amélioration identifié pour une version future.

Base de Donnees

Vue d'ensemble

La base de données est la clé de voûte de l'application Smart Campus. Nous utilisons **MariaDB** avec l'ORM **Doctrine**. Mon rôle a été de garantir la cohérence des données et d'optimiser les performances pour l'affichage en temps réel.

---

1. Conception du Modèle de Données

Justification des choix de conception

Le modèle de données a été conçu pour refléter la logique métier du projet Smart Campus. Voici les entités principales et les choix de conception :

Entité `SystemeAcquisition` (SA) :

- **Choix :** Une seule propriété `etat` plutôt que deux propriétés séparées (position + état technique)
- **Justification :** Comme expliqué dans la partie Développement Web Full Stack, la fusion en une **Machine à États Finis** simplifie la logique métier et évite les incohérences (ex: un SA "en stock" mais "installé")

Relations entre entités :

Relation	Type	Justification
SA → Salle	ManyToOne	Un SA peut être installé dans une seule salle, mais une salle peut contenir plusieurs SA
SA → Demande	OneToMany	Un SA peut avoir plusieurs demandes dans son historique (installation, désinstallation)
Demande → Technicien	ManyToOne	Une demande est assignée à un technicien

---

2. Implantation et Contraintes de Validation

Respect de la conception

L'implémentation Doctrine respecte fidèlement le modèle conceptuel. Chaque entité, attribut et relation a été implémenté avec les annotations/attributs PHP 8.

Exemple d'entité avec ses relations :

```php
#[ORM\Entity(repositoryClass: SystemeAcquisitionRepository::class)]
class SystemeAcquisition
{
[...]

    public function getEtat(): ?string
    {
        return $this->etat;
    }

    public function setEtat(?string $etat): static
    {
        $this->etat = $etat;

        return $this;
    }

[...]
}
```

Règles de validation personnalisées

Au-delà des contraintes SQL standards (NOT NULL, types), j'ai mis en place des **validations métier** :

Validation dans le Service (Logique métier) :

```
// SystemeAcquisitionManager.php




public function changeEtat(SystemeAcquisition $sa, string $nouvelEtat):
array
{
    //  Validation personnalisée : liste blanche des états autorisés
    $allowedEtats = [
        'installé-opérationnel', 'installé-hs',
        'stock-disponible', 'stock-hs',
        'à-installer', 'à-désinstaller'
    ];

    if (!in_array($nouvelEtat, $allowedEtats)) {
        return ['success' => false, 'message' => 'État invalide.'];
    }
    // ...
}

public function deleteSystemeAcquisition(SystemeAcquisition $sa): array
{
    //  Règle métier : impossible de supprimer un SA installé
    if ($sa->getEtat() !== 'stock-disponible' && $sa->getEtat() !==
'stock-hs') {
        return ['success' => false, 'message' => 'Le SA doit être en stock
pour être supprimé.'];
    }
    // ...
}
```

Validation dans les Formulaires (Symfony Validator) :

```
// SaGenerationType.php




$builder->add('quantite', IntegerType::class, [
    'constraints' => [
        new NotBlank(),
```

```
        new Range(['min' => 1, 'max' => 50, 'notInRangeMessage' => 'Entre
1 et 50']),
    ],
]);
```

---

3. Évolution du Modèle : Migrations

Gestion des évolutions avec Doctrine Migrations

Toutes les évolutions du schéma de base de données sont gérées via le système de **migrations Doctrine**. Cela permet :

- Un **versionnement** des changements de structure
- Une **reproductibilité** sur tous les environnements (dev, prod)
- Un **historique** des modifications

Commandes utilisées :

```
# Générer une migration après modification d'une entité
php bin/console make:migration

# Appliquer les migrations en attente
php bin/console doctrine:migrations:migrate
```

Exemple de migration générée :

```
// migrations/Version20250110123456.php




public function up(Schema $schema): void
{
    $this->addSql('ALTER TABLE systeme_acquisition ADD nom_sa VARCHAR(100)
DEFAULT NULL');
}

public function down(Schema $schema): void
{
    $this->addSql('ALTER TABLE systeme_acquisition DROP nom_sa');
}
```

Robustesse :

- Chaque migration possède une méthode `up()` (application) et `down()` (rollback)
- Les migrations sont exécutées dans l'ordre chronologique

- Un développeur peut réinstaller le projet complet avec `doctrine:migrations:migrate`

---

4. Requêtes Optimisées

Le problème du "N+1" requêtes

Sur le tableau de bord technicien, nous devons afficher des statistiques globales (ex: "Combien de SA sont actuellement HS ?"). Une approche naïve aurait été de récupérer tous les systèmes en PHP (`findAll()`) puis de les compter un par un dans une boucle. Cela aurait entraîné une consommation mémoire excessive et des ralentissements avec l'augmentation du parc.

Ma solution (QueryBuilder) : J'ai utilisé le **QueryBuilder** de Doctrine pour déléguer le calcul directement au moteur de base de données via une requête SQL agrégée (`GROUP BY`).

Extrait de code (Repository) :

```
// Repository/SystemeAcquisitionRepository.php




/**



 * Récupère les stats par état en une seule requête optimisée
 */
public function getStatisticsByEtat(): array
{
    return $this->createQueryBuilder('sa')
        ->select('sa.etat, COUNT(sa.id) as count') // Calcul SQL
        ->groupBy('sa.etat')
        ->getQuery()
        ->getResult();
}
```

Gain : On passe de plusieurs centaines d'objets hydratés en mémoire à un simple tableau de résultats légers. Le temps de chargement de la page est instantané, quelle que soit la taille du parc.

---

5. Fixtures et Simulation de scénarios complexes

Pour la Release 2, nous devions présenter une démo au client sans avoir encore déployé de capteurs physiques. Il ne suffisait pas de créer des données simples ; il fallait simuler des **situations métier cohérentes** pour tester l'interface.

Défi technique (Relations ORM) : J'ai dû gérer des dépendances multiples pour créer un historique crédible. Par exemple, pour tester la fonctionnalité "Désinstallation", il me fallait un SA qui soit *déjà* installé dans une salle, avec une demande d'installation *passée* et clôturée.

Extrait de code (DataFixtures) :

```
// Simulation d'un cycle de vie complet pour le test




// 1. Création de l'environnement (La Salle)



$salle = new Salle();
$salle->setNomSalle('Salle-Test-Desinstallation');
$manager->persist($salle);

// 2. Création du SA (Lié à la salle, état "à désinstaller")



$sa = new SystemeAcquisition();
$sa->setEtat('à-désinstaller');
$sa->setSalle($salle); // Relation ManyToOne
$manager->persist($sa);

// 3. Historique : Ancienne demande d'installation (Clôturée)



$ancienneDemande = new Demande();
$ancienneDemande->setTypeDemande('installation');
$ancienneDemande->setStatut('Terminé');
$ancienneDemande->setSystemeAcquisition($sa);
$manager->persist($ancienneDemande);

// 4. Actuel : Demande de désinstallation (En cours)



$nouvelleDemande = new Demande();
$nouvelleDemande->setTypeDemande('désinstallation');
$nouvelleDemande->setStatut('En cours');
$nouvelleDemande->setSystemeAcquisition($sa);
$manager->persist($nouvelleDemande);
```

Analyse réflexive : L'écriture de ces fixtures m'a forcé à comprendre finement le modèle relationnel de notre application. Elles ont servi de socle à toute l'équipe pour développer les vues Frontend sans attendre les données réelles.

---

6. Tests et Couverture

Tests sur les entités et la base de données

Les tests PHPUnit couvrent également la couche données :

Tests du Repository (Requêtes) :

- Vérification que `getStatisticsByEtat()` retourne les bonnes statistiques
- Tests des méthodes `findBy()` personnalisées

Tests du Service (Logique métier liée aux données) :

- Création de SA avec persistance en base
- Changement d'état avec validation
- Suppression conditionnelle (règle métier)

Couverture des tests :

La couverture globale du projet est de **77.65%** (détaillée dans la section Développement Web Full Stack). Cette couverture inclut les tests sur :

- Les entités et leurs relations
- Les repositories et leurs requêtes optimisées
- Les services manipulant les données

Axe d'amélioration : Pour atteindre les 80%+ recommandés, il faudrait ajouter des tests unitaires spécifiques pour :

- Chaque méthode du Repository
- Les cas limites des validations (entités avec données invalides)
- Les migrations (tests de rollback)

Bilan et Perspectives

Synthèse des compétences

Compétence	Points forts	Axes d'amélioration
Gestion de projet	Story Mapping, DOR/DOD, cérémonies Agile, cohésion d'équipe	Burndown Chart, Merge Requests formalisées
Web Full Stack	Architecture Services, Tests PHPUnit, Formulaires Symfony	Responsive mobile, Events/Listeners

Compétence	Points forts	Axes d'amélioration
Base de données	QueryBuilder optimisé, Fixtures complexes, Migrations	Couverture tests >80%
Embarqué	Compréhension architecture, FreeRTOS basique	Contribution limitée (5%)

Ce que ce projet m'a apporté

Compétences techniques :

- Maîtrise de l'architecture Symfony (Services, Repositories, Forms)
- Compréhension de l'ORM Doctrine et de l'optimisation SQL
- Initiation au développement embarqué (FreeRTOS, ESP32)

Compétences transversales :

- Expérience en tant que Product Owner : négociation client, priorisation, validation
- Animation d'équipe : Daily Meetings, rétrospectives, gestion des conflits
- Rigueur méthodologique : DOR/DOD, traçabilité, documentation

Perspectives pour la suite

Court terme :

- Compléter les TODO de ce portfolio (captures d'écran, diagrammes, liens)
- Atteindre les 80% de couverture de tests

Moyen terme :

- Mettre en place les Merge Requests et le Burndown Chart sur le prochain projet
- Approfondir le développement embarqué si l'occasion se présente

Long terme :

- Consolider mon profil de développeur Web Full Stack avec des compétences en gestion de projet
- Explorer d'autres frameworks (React, Vue.js) pour compléter ma stack technique

---